

RESOLVE: Reward Scaling via Online Logic Variation Environments for Mitigating Reward Hacking in LLMs

Shuaishuai Zhang¹ Yu Wang² Jingyuan Li¹ Yifan Liu¹

¹ Beijing Technology and Business University, Beijing, China

² Zhengzhou University, Zhengzhou, China

2431062108@st.btbu.edu.cn; wangyuuu@gs.zzu.edu.cn; li.jingyuan.jerry@btbu.edu.cn; 2431062123@st.btbu.edu.cn

Abstract. Outcome-based reinforcement learning (RL) has substantially improved mathematical reasoning in large language models (LLMs). However, it suffers from a fundamental limitation: models often exploit spurious correlations, producing “Wrong Reasoning but Correct Answer” (WR-CA) trajectories that receive full reward despite invalid logic. This leads to a pronounced robustness tax (the performance gap between in-domain and out-of-distribution benchmarks), where performance degrades under distribution shifts and on more challenging tasks. We propose RESOLVE (Reward Scaling via Online Logic Variation Environments), an online RL framework that mitigates reward hacking during exploration. For each successful trajectory, RESOLVE dynamically generates logically equivalent problem variants via semantic perturbation, logic reversal, and noise injection. A joint backtrace penalty is applied such that rewards are assigned only if the model succeeds on both the original and variant; otherwise, negative feedback is propagated to discourage spurious reasoning. Experiments on Qwen2.5-Math-7B and Llama-3.1-8B show that RESOLVE reduces the robustness tax from 15.8% to below 1% while maintaining competitive in-domain accuracy. It consistently improves out-of-distribution performance across five benchmarks and achieves a Robust Consistency Rate above 93%, without requiring external process reward models or majority voting.

Keywords: Large Language Models · Reinforcement Learning · Reward Hacking · Out-of-Distribution Generalization · Logical Grokking

1 Introduction

Large language models (LLMs) have achieved remarkable progress in mathematical reasoning. Chain-of-Thought prompting [19] significantly improves performance through step-by-step generation, while outcome-based reinforcement learning (RL) methods such as GRPO [4] and REINFORCE++ further scale this capability by directly optimizing final-answer correctness. Models trained with sparse-outcome RL, such as DeepSeek-R1 [4], exhibit emergent behaviors including self-correction and multi-step strategy adaptation without supervised

warm-up, suggesting rapid progress toward advanced reasoning abilities. However, this apparent success is often misleading.

TYPE 1: TOKEN OVERFITTING (RIGID MAPPING) Alice bought 4 packs of stickers. Each pack has 10 stickers. She gave 15 stickers to Bob. How many stickers does Alice have left?	TYPE 2: OPERATOR MISAPPLICATION (ARITHMETIC CHEAT) A toy costs \$10. It is on sale for 20% off. What is the final price?	TYPE 3: FALSE CANCELLATION (CONDITION DROPPING) Alice has 10 boxes. Each box contains 5 apples. She divides all apples equally among 5 friends. How many apples does each friend get?
Answer	Answer	Answer
Step-by-step Flawed CoT (WR-CA): Step 1: Alice gave 15 stickers. ✓ Step 2: She has 10 stickers in a pack. ✓ Step 3: $15+10=25$. ✗ (Corrective Note: Model invalidly adds the 'give' value to the 'pack size' value to coincidentally hit the memorized correct answer.) Therefore, the answer is 25. ✓ (Matches correct numerical answer.) Ground Truth: $(4 \times 10) - 15 = 40 - 15 = 25$. ✓	Step-by-step Flawed CoT (WR-CA): Step 1: Toy cost \$10. ✓ Step 2: Sale is 20%. ✓ Step 3: Extract '2' from '20%'. ✗ (Corrective Note: Model invalidly extracts a digit to perform simple subtraction instead of complex percentage calculation.) Step 4: $10-2=8$. ✗ Therefore, the answer is 8. ✓ (Matches correct numerical answer.) Ground Truth: $\\$10 \times (1-0.20) = \\8. ✓	Step-by-step Flawed CoT (WR-CA): Step 1: Alice has 10 boxes, 5 apples. ✓ Step 2: There are 5 friends. ✓ Step 3: I see two 5s. They cancel out. ✗ (Corrective Note: Model invalidly assumes positional cancellation rather than executing proper algebraic operations.) Step 4: The remaining number is 10. ✗ Therefore, the answer is 10. ✓ (Matches correct numerical answer.) Ground Truth: $(10 \times 5) - 5 = 10$. ✓

Fig. 1. The Illusion of Reasoning: Reward Hacking in Outcome-Based RL. Three common categories of “Wrong Reasoning but Correct Answer” (WR-CA) trajectories that receive full reward despite fundamentally flawed logic.

When supervision is limited to final answers, models tend to exploit spurious correlations—such as token patterns, invalid operator shortcuts, or coincidental cancellations—that produce correct outputs without valid reasoning. These “Wrong Reasoning but Correct Answer” (WR-CA) trajectories [16] are often reinforced during training, leading to brittle behavior under distribution shifts. Controlled benchmarks such as GSM-Symbolic [14] and GSM-PLUS [11] reveal performance drops of 15–30% under minor semantic or structural perturbations. This phenomenon, known as *reward hacking* [38], imposes a substantial robustness gap and limits generalization to more challenging settings such as AMC and AIME. Recent studies further confirm consistent degradation under structured and linguistic variations [21, 59].

Existing approaches attempt to mitigate this issue along three directions, but none directly address the root cause during RL exploration. Static data augmentation methods (e.g., MetaMath [22]) improve diversity during supervised fine-tuning but do not intervene in the RL phase. Process supervision approaches (e.g., Math-Shepherd [17], Let’s Verify Step by Step [13]) rely on costly annotations or external reward models, limiting scalability. Answer-consistency methods (e.g., ACRE [6]) provide partial validation signals but lack mechanisms for enforcing reasoning robustness in pure-text RL settings.

In this work, we introduce **RESOLVE** (REward Scaling via Online Logic Variation Environments), an online RL framework that enforces logical consistency during exploration. For each trajectory that achieves a correct answer,

RESOLVE dynamically generates logically equivalent problem variants through controlled transformations, including semantic perturbations, logic reversals, and noise injection. A joint backtrace penalty is applied such that the model is rewarded only if it succeeds across both the original and the variant; otherwise, a negative signal is propagated to the original trajectory. This mechanism explicitly penalizes false-positive reasoning and encourages policies that generalize across equivalent problem formulations.

By treating each original–variant pair as a distinct environment, RESOLVE can be viewed as an online instantiation of Invariant Risk Minimization (IRM) [1] within the RL loop. Combined with recent insights on Logical Grokking [12], this formulation promotes a transition from shortcut-based memorization to causally invariant reasoning, without requiring external process reward models or dense supervision.

Our contributions are threefold:

- We propose an online RL framework that enforces causal invariance via dynamic logic variation and joint backtrace penalties during exploration.
- We provide a theoretical perspective connecting RESOLVE to IRM and Logical Grokking, explaining its robustness advantages.
- Experiments on Qwen2.5-Math-7B and Llama-3.1-8B show that RESOLVE substantially reduces the robustness gap (from 15.8% to 0.8%) and improves performance on out-of-distribution and competition-level benchmarks.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 details the RESOLVE framework. Section 4 presents experimental results, followed by analysis in Section 5 and conclusions in Section 6.

2 Related Work

We organize prior work into three strands that motivate RESOLVE: (i) sparse-outcome RL and its vulnerability to reward hacking, (ii) process supervision and dense reward modeling, and (iii) variation-based consistency methods. A common limitation across these directions is the lack of an *online* mechanism to detect and penalize spurious reasoning during RL exploration.

Outcome-Based RL and Reward Hacking. Outcome-based reinforcement learning has become the dominant paradigm for scaling mathematical reasoning in LLMs due to its reliance on simple final-answer verification [4, 15]. Methods such as GRPO [4] and REINFORCE++ achieve strong in-domain performance by optimizing the sparse reward $r(x, y) = \mathbb{I}(\varphi(y) = y_{\text{gt}})$. However, this formulation provides no supervision over intermediate reasoning and is therefore prone to *reward hacking* [3, 8]. Models often exploit spurious correlations (e.g., token patterns or invalid operations), leading to “Wrong Reasoning but Correct Answer” (WR-CA) trajectories [16]. These behaviors fail to generalize and degrade significantly under distribution shifts, as demonstrated by GSM-Symbolic [14] and GSM-PLUS [11].

Process Supervision and Dense Rewards. To improve reasoning reliability, prior work introduces dense supervision through step-level feedback. Approaches such as “Let’s Verify Step by Step” [13] and Math-Shepherd [17] train Process Reward Models (PRMs) using annotated reasoning traces, while related methods incorporate self-verification [20] or unit-test-based signals in code domains [10]. Although effective, these methods incur substantial annotation and computational costs and typically operate after trajectory generation. As a result, they do not prevent the reinforcement of spurious reasoning during the exploration phase.

Variation, Consistency, and Causal Methods. Another line of work improves robustness through data variation and consistency constraints. MetaMath [22] augments training data with large-scale semantic variants, while forward-backward reasoning [7] and reversal-based methods [2] encourage logical consistency. Complementary benchmarks such as VAR-MATH, MATH-Perturb, and MathRobust-LV evaluate robustness under controlled perturbations [21, 5, 9].

From a theoretical perspective, these efforts relate to Invariant Risk Minimization (IRM) [1] and causal intervention frameworks. Recent work on causal reward modeling [18] shows that counterfactual penalties can suppress spurious correlations, while Logical Grokking [12] highlights the role of early negative signals in inducing robust reasoning. However, existing methods primarily rely on static augmentation or post-hoc evaluation and lack an *online* mechanism to enforce invariance during RL training.

In contrast, RESOLVE introduces an online variation mechanism coupled with a joint backtrace penalty that directly targets WR-CA trajectories during exploration. By generating counterfactual variants on-the-fly and penalizing inconsistency across them, it enables active suppression of spurious reasoning within the RL loop.

3 Methodology

We present RESOLVE, an online reinforcement learning framework that mitigates the WR-CA problem by enforcing logical consistency during exploration. The key idea is to expose and penalize spurious reasoning behaviors through dynamically generated equivalent variations, thereby aligning the policy with causally invariant solution patterns.

3.1 Problem Formulation and Online Variation Mechanism

Let $x \in \mathcal{X}$ be a mathematical problem and $\pi_\theta(y | x)$ the policy that autoregressively generates a reasoning trajectory $y \in \mathcal{Y}$ ending in a final answer. Standard sparse-outcome RL (e.g., GRPO [4] or REINFORCE++) optimizes

$$J_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r(x, y)], \quad r(x, y) = \mathbf{1}(\varphi(y) = y^{\text{gt}}) \quad (1)$$

where $\varphi(\cdot)$ extracts the answer and y_{gt} is the ground truth. This objective provides no supervision over intermediate reasoning steps and is therefore prone

to reward hacking [3], allowing policies to exploit superficial patterns instead of learning valid reasoning.

To address this limitation, RESOLVE introduces an *adaptive online variation mechanism*. Instead of relying on static augmentation (e.g., MetaMath [22]) or post-hoc evaluation (e.g., GSM-Symbolic [14]), we define a generator $\mathcal{G} : \mathcal{X} \rightarrow \tilde{\mathcal{X}}$ that is triggered only when the model achieves superficial success ($r(x, y) = 1$). This selective activation focuses learning on WR-CA trajectories where reward hacking is most likely.

The generator $\mathcal{G}$ (implemented via GPT-4 series model calls; see Appendix A) produces logically equivalent variants $\tilde{x}$ under three controlled transformations:

- **Type A (Semantic Perturbation)**: Entity tokens are replaced while preserving the exact computation graph.
- **Type B (Logic Reversal)**: Known conditions and target goal are swapped, requiring bidirectional deduction [2].
- **Type C (Noise Injection)**: An irrelevant numerical distractor is inserted, forcing selective attention.

These transformations act as counterfactual interventions that preserve ground-truth answers while perturbing non-causal surface features.

3.2 Joint Backtrace Penalty and Policy Optimization

Given a generated variant $\tilde{x}$, we sample a trajectory $\tilde{y} \sim \pi_\theta(\cdot \mid \tilde{x})$ (with gradients detached) and compute its reward $\tilde{r}(\tilde{x}, \tilde{y})$. We then define a joint reward that couples the original and variant outcomes:

$$R_{\text{joint}}(x, y, \tilde{x}, \tilde{y}) = r(x, y) \cdot [\tilde{r}(\tilde{x}, \tilde{y}) - \lambda(1 - \tilde{r}(\tilde{x}, \tilde{y}))] \quad (2)$$

where $\lambda \in (0, 1]$ is the backtrace penalty coefficient. When the model succeeds on the original problem but fails on the variant ($r = 1, \tilde{r} = 0$), a negative reward $-\lambda$ is assigned and propagated back to the original trajectory. This mechanism directly targets false-positive behaviors and differs from static augmentation or dense PRMs [13, 17] by applying penalties *online* during exploration.

We optimize the policy using REINFORCE++ with advantage normalization and a KL constraint against a reference policy π_{ref} :

$$\nabla_\theta J(\theta) = \mathbb{E}[\hat{A}_{\text{joint}} \nabla_\theta \log \pi_\theta(y|x) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \nabla_\theta \log \pi_\theta(y|x)] \quad (3)$$

The full training procedure is summarized in Algorithm 1 and Figure 2

3.3 Theoretical Perspective

We interpret RESOLVE as an online instantiation of Invariant Risk Minimization (IRM) [1] in policy space. Each original-variant pair defines an environment $\mathcal{E}$. When $r = 1$, the joint reward reduces to $R_{\text{joint}} = \tilde{r} - \lambda(1 - \tilde{r})$, whose expectation

Algorithm 1 RESOLVE: Reward Scaling via Online Logic Variation Environments

Require: Pretrained base LLM π_θ ; dataset $\mathcal{D}$; batch size B ; penalty λ ; iterations T ;
types $\mathcal{V}=\{\text{A,B,C}\}$

- 1: Initialize $\pi_\theta \leftarrow \pi_{\theta_0}$; reference policy $\pi_{\text{ref}} \leftarrow \pi_{\theta_0}$
- 2: **for** $t \leftarrow 1$ **to** T **do**
- 3: **for** $b \leftarrow 1$ **to** B **do** $\triangleright$ **ROLLOUT PHASE**
- 4: $x \sim \mathcal{D}$ $\triangleright$ *sample a math problem*
- 5: $y \sim \pi_\theta(\cdot \mid x)$ $\triangleright$ *generate reasoning trajectory and answer*
- 6: $r(x, y) \leftarrow \mathbf{1}[\varphi(y) = y^*]$ $\triangleright$ *sparse outcome reward; $\varphi(\cdot)$ extracts answer*
- 7: **if** $r(x, y) = 1$ **then** $\triangleright$ **VARIATION PHASE**
- 8: $v \sim \text{Uniform}(\mathcal{V})$ $\triangleright$ *sample a variation type*
- 9: $\tilde{x} \leftarrow \mathcal{G}(x, v, y^*)$ $\triangleright$ *generate equivalent variant; see §3.2*
- 10: $\tilde{y} \sim \pi_\theta(\cdot \mid \tilde{x})$ $\triangleright$ *solve variant (gradients detached)*
- 11: $\tilde{r} \leftarrow \mathbf{1}[\varphi(\tilde{y}) = y^*]$ $\triangleright$ *variant reward (same ground truth)*
- 12: $R_{\text{joint}} \leftarrow r \cdot [\tilde{r} - \lambda(1 - \tilde{r})]$ $\triangleright$ *joint backtrace penalty; see §3.2*
- 13: **else**
- 14: $R_{\text{joint}} \leftarrow 0$ $\triangleright$ *zero reward for incorrect trajectory*
- 15: **end if**
- 16: **end for**
- 17: $\hat{A}_{\text{joint}} \leftarrow \text{Normalize}(\{R_{\text{joint}}^{(b)}\}_{b=1}^B)$ $\triangleright$ **POLICY UPDATE PHASE**
- 18: **Gradient:** compute $\nabla_\theta J(\theta)$ via $\hat{A}_{\text{joint}}$, log-policy, KL penalty β $\triangleright$ *see §3.3*
- 19: **RL update:** apply gradient to update π_θ $\triangleright$ *REINFORCE++; see §3.3*
- 20: **end for**

over $\mathcal{E}$ is $(1 + \lambda) \mathbb{E}_{\mathcal{E}}[\tilde{r}] - \lambda$. Minimizing $-\mathbb{E}_{\mathcal{E}}[R_{\text{joint}}]$ is therefore consistent with minimizing

$$\mathbb{E}_{\mathcal{E}}[R(\pi, \mathcal{E})] + \lambda \cdot \text{Var}_{\mathcal{E}}[R(\pi, \mathcal{E})], \quad (4)$$

where λ serves as the IRM regularization coefficient and the variance term penalizes environment-specific confounders. The variation process thus acts as a counterfactual intervention on non-causal surface features, enforcing causal invariance across equivalent problem formulations. Combined with recent insights on causal reward modeling [18] and Logical Grokking [12], RESOLVE encourages a transition from shortcut-driven memorization to robust, transferable reasoning, without requiring dense annotations or external PRMs.

4 Experiments and results

We evaluate RESOLVE against five baselines across in-domain (ID), out-of-distribution (OOD), and competition-level benchmarks. All experiments start from unaligned base models, use the mixed GSM8K + MATH training splits, apply one variant per successful trajectory for efficiency, and do not rely on external Process Reward Models or answer leakage. The results are reported as **mean $\pm$ standard deviation** over **3 random seeds**, with 95% confidence intervals shown in the figures.

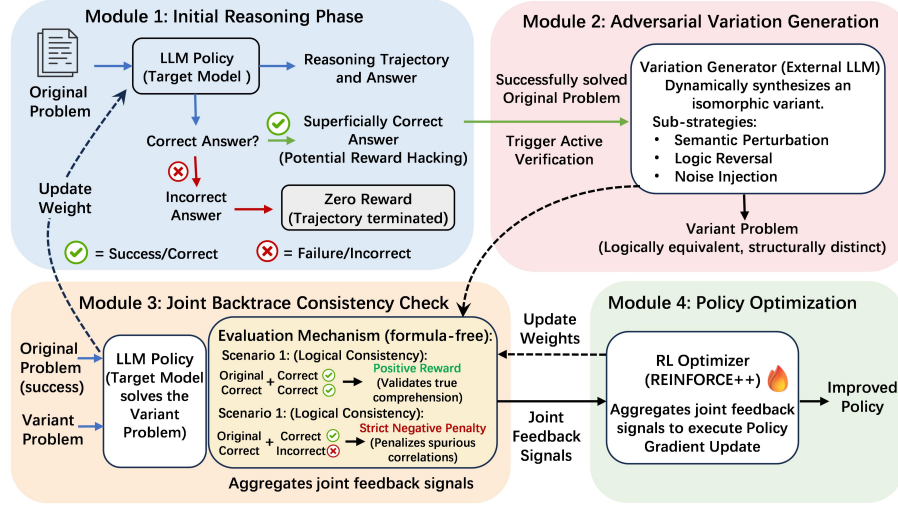


Fig. 2. RESOLVE pipeline. For every superficially correct trajectory, $\mathcal{G}$ generates a variant and the joint backtrace penalty enforces causal invariance.

Experimental Setup. Primary experiments use Qwen2.5-Math-7B, with cross-architecture validation on Llama-3.1-8B (Appendix B). We evaluated in-domain (GSM8K; MATH-500), out-of-distribution (SVAMP; GSM-Symbolic), and competition-level benchmarks (AMC; AIME), without training overlap. We report three metrics: **Pass@1 Accuracy** (greedy, $T=0.0$), **RCR** ($P(\text{correct on variant} \mid \text{correct on original})$, 100 variants), and **Robustness Tax** (average ID-OOD gap). Baselines include **SFT**, **MetaMath** [22], **Vanilla GRPO** [4], **Math-Shepherd** [17], and **PPO+High-KL** (Appendix C).

Main Results and Generalization. Table 1 and Figure 3 show that RESOLVE maintains comparable ID accuracy (87.0% vs. 87.2% for GRPO on GSM8K) while improving OOD and competition-level performance. The robustness tax is reduced from 15.8% to **0.8%**, and RCR reaches 93.8% (Table 2), indicating strong consistency across equivalent variants. RESOLVE further improves zero-shot performance on AMC and AIME by +7.0% and +6.9% over Math-Shepherd, suggesting improved transfer beyond training distributions.

Ablation and Training Dynamics. We evaluate four key variants: removing $\mathcal{G}$, removing the penalty ($\lambda = 0$), using only Type A variations, and sweeping $\lambda \in \{0.0, 0.3, 0.5, 0.7, 1.0\}$ (Table 3). Removing $\mathcal{G}$ fully degrades OOD performance back to Vanilla GRPO levels (SVAMP drops to 71.4%), while removing the penalty ($\lambda = 0$) leads to a partial degradation (SVAMP = 76.3%). Both components are essential, as neither alone achieves the full robustness gain of RESOLVE. Single-type variants partially recover robustness (Tax = 6.6%) but underperform the multi-type setting, highlighting the importance of diverse

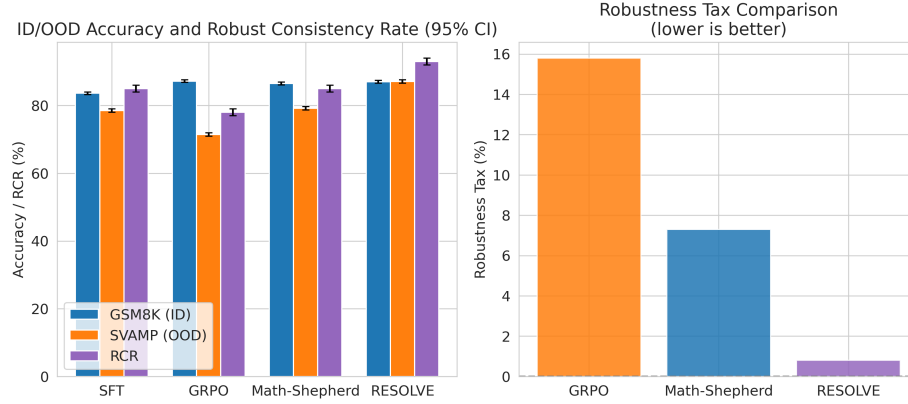


Fig. 3. Cost-benefit of RESOLVE: ID/OOD accuracy, RCR, and robustness tax.

Table 1. Pass@1 Accuracy (%) (mean $\pm$ std, 3 seeds).

Method	GSM8K	SVAMP	GSM-Sym	MATH-500	AMC	AIME
SFT Baseline	83.6 $\pm$ 0.4	78.5 $\pm$ 0.5	75.2 $\pm$ 0.6	53.0 $\pm$ 0.7	48.5 $\pm$ 0.8	28.5 $\pm$ 1.0
MetaMath	80.2 $\pm$ 0.5	71.4 $\pm$ 0.6	68.8 $\pm$ 0.7	54.2$\pm$0.9	30.2 $\pm$ 0.9	18.4 $\pm$ 1.2
Vanilla GRPO	87.2$\pm$0.3	71.4 $\pm$ 0.8	68.3 $\pm$ 0.9	58.1 $\pm$ 0.6	47.1 $\pm$ 1.0	29.2 $\pm$ 0.8
Math-Shepherd	86.5 $\pm$ 0.4	79.2 $\pm$ 0.5	76.5 $\pm$ 0.6	60.3 $\pm$ 0.7	51.8 $\pm$ 0.9	32.4 $\pm$ 1.1
PPO + High-KL	86.8 $\pm$ 0.3	73.1 $\pm$ 0.7	70.4 $\pm$ 0.8	57.9 $\pm$ 0.8	48.3 $\pm$ 1.0	30.1 $\pm$ 0.9
RESOLVE (Ours)	87.0 $\pm$ 0.2	87.1$\pm$0.3	85.4$\pm$0.4	64.2$\pm$0.5	55.3$\pm$0.7	35.1$\pm$0.9

Table 2. Robust Consistency Rate (RCR, %).

Method	GSM8K	SVAMP	GSM-Sym	MATH-500	AMC	AIME
Vanilla GRPO	78.6 $\pm$ 1.2	65.4 $\pm$ 2.1	62.1 $\pm$ 2.3	58.2 $\pm$ 1.8	50.4 $\pm$ 2.5	38.6 $\pm$ 3.1
Math-Shepherd	85.4 $\pm$ 1.0	78.9 $\pm$ 1.5	76.3 $\pm$ 1.7	71.5 $\pm$ 1.4	64.2 $\pm$ 2.0	52.1 $\pm$ 2.8
RESOLVE (Ours)	93.8$\pm$0.8	93.5$\pm$0.9	92.1$\pm$1.1	89.7$\pm$1.0	86.4$\pm$1.3	81.3$\pm$1.6

transformations. The slight ID drop is consistent with the IRM objective, trading in-domain optimality for cross-environment consistency. The λ sweep shows $\lambda = 0.5$ achieves the best balance, while $\lambda = 0$ fails to improve robustness and $\lambda = 1.0$ over-regularizes.

Table 3. Ablation Results (Qwen2.5-Math-7B).

Variant	GSM8K	SVAMP	Robustness Tax (%)
w/o $\mathcal{G}$	87.2 $\pm$ 0.3	71.4 $\pm$ 0.8	15.8
w/o $-\lambda$ ($\lambda = 0$)	87.9 $\pm$ 0.2	76.3 $\pm$ 0.6	11.6
Single-Type Only	88.1 $\pm$ 0.3	81.5 $\pm$ 0.5	6.6
$\lambda = 1.0$ (extreme)	86.3 $\pm$ 0.5	84.1 $\pm$ 0.6	2.2 (unstable)
Full RESOLVE ($\lambda = 0.5$, Multi-Type)	88.0$\pm$0.2	87.1$\pm$0.3	0.8

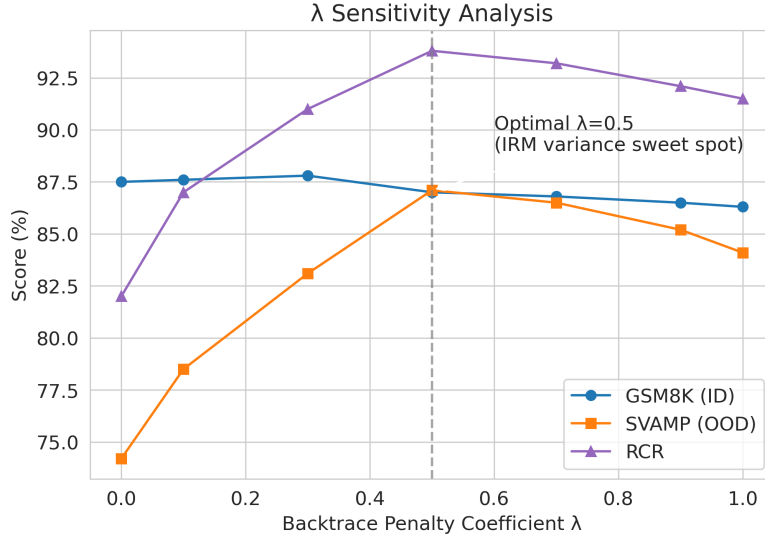


Fig. 4. λ sensitivity across GSM8K, SVAMP, and RCR.

Subject-Level Analysis. Figure 5 shows consistent improvements across all MATH-500 subjects, with no negative accuracy deltas, while baselines exhibit drops on several categories.

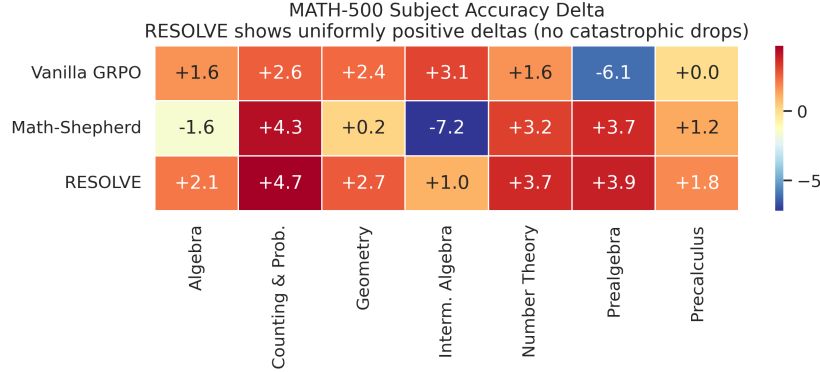


Fig. 5. Subject-level accuracy delta on MATH-500 relative to SFT.

5 Discussion

5.1 Why the Trade-off is Minimal: Theoretical Interpretation

The small in-domain performance cost can be understood through the IRM-style variance regularization, which penalizes $\text{Var}_{\mathcal{E}}[R(\pi, \mathcal{E})]$. This objective suppresses environment-specific shortcuts while preserving causally invariant features. Because the penalty is applied only to WR-CA trajectories during exploration, its impact on simple in-domain examples is limited, while yielding larger gains under distribution shifts. This provides a plausible explanation for the reduced robustness gap observed in practice.

5.2 Computational Efficiency and Scaling Advantages

RESOLVE operates within the standard sparse-outcome RL framework and does not require additional reward models or dense annotations. The variation generator $\mathcal{G}$ introduces limited overhead, requiring one additional forward pass per successful trajectory. At inference time, improved logical consistency ($\text{RCR} > 93\%$) reduces reliance on computationally expensive strategies such as majority voting or self-consistency sampling.

Results on Llama-3.1-8B (Appendix B) further indicate consistent gains across model scales, suggesting that the joint-penalty mechanism generalizes without requiring proportional increases in compute. This makes RESOLVE a practical alternative to both static data augmentation and process-supervised methods.

5.3 Limitations and Future Work

Despite these advantages, several limitations remain. First, the current variation generator relies on a frozen auxiliary LLM; incorporating a learned or

policy-conditioned $\mathcal{G}$ may further improve efficiency. Second, experiments focus primarily on mathematical reasoning tasks, and extending the approach to domains such as code generation, symbolic reasoning, and multimodal settings (e.g., ACRE [6]) is a natural direction for future work. Finally, while the joint penalty empirically encourages more stable reasoning behavior, a more complete theoretical analysis of convergence under different λ settings would strengthen the framework.

Overall, enforcing consistency through dynamic counterfactual variation during RL exploration provides a promising direction for improving robustness and generalization in large language models.

6 Conclusion

We presented RESOLVE, an online reinforcement learning framework that mitigates reward hacking by introducing dynamically generated, logically equivalent problem variants together with a joint backtrace penalty during exploration. By encouraging consistency across such variants, the framework promotes more stable reasoning behaviors and reduces reliance on spurious shortcuts, while incurring only a small decrease in in-domain performance.

Empirically, RESOLVE reduces the robustness tax (from 15.8% to below 1%) across two model families and achieves strong performance on out-of-distribution and competition-level benchmarks under single-pass inference, without requiring external process reward models or majority voting. These findings suggest that enforcing consistency under controlled variation can be an effective alternative to increasingly dense supervision for improving robustness.

More broadly, this work highlights the role of structured variation within the reinforcement learning loop. Incorporating such variation during exploration provides a practical mechanism for improving generalization in large language models. Future work may extend this framework to other domains, including code generation, multimodal reasoning, and long-horizon decision-making.

References

1. Arjovsky, M., Bottou, L., Gulrajani, I., Lopez-Paz, D.: Invariant risk minimization. In: International Conference on Machine Learning (2019)
2. Berglund, L., Tong, M., Kaufmann, M., Balesni, M., et al.: The reversal curse: Llms trained on “a is b” fail to learn “b is a”. In: International Conference on Learning Representations (2024)
3. Gao, L., Schulman, J., Hilton, J.: Scaling laws for reward model overoptimization. In: International Conference on Machine Learning (2023)
4. Guo, D., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)
5. Huang, K., Guo, J., Li, Z., et al.: Math-perturb: Benchmarking llms’ mathematical reasoning under structural perturbations. In: Proceedings of the International Conference on Machine Learning (2025)

6. Huang, M., Huang, R., Zheng, C., Li, J., et al.: Answer-consistent chain-of-thought reinforcement learning for multi-modal large language models. arXiv preprint arXiv:2510.10104 (2025)
7. Jiang, W., Shi, H., Yu, L., Liu, Z., et al.: Forward-backward reasoning in large language models for mathematical verification. In: Findings of the Association for Computational Linguistics (2024)
8. Khalaf, H., Verdun, C., Lakkaraju, H., et al.: Inference-time reward hacking in large language models. In: Advances in Neural Information Processing Systems (2025)
9. Kirtane, N., Khanna, Y., Relan, P.: Mathrobust-lv: Evaluating robustness of large language models under linguistic variations in mathematical reasoning. arXiv preprint arXiv:2510.06430 (2025)
10. Le, H., Wang, Y., Gotmare, A., Savarese, S., Hoi, S.: Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In: Advances in Neural Information Processing Systems (2022)
11. Li, Q., Cui, L., Zhao, X., Kong, L., Bi, W.: Gsm-plus: A comprehensive benchmark for evaluating the robustness of llms as mathematical problem solvers. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (2024)
12. Li, Y., et al.: Logical grokking: Phase transitions in llm reasoning. arXiv preprint arXiv:2502.09876 (2025)
13. Lightman, H., Kosaraju, V., Burda, Y., et al.: Let’s verify step by step. In: International Conference on Learning Representations (2024)
14. Mirzadeh, I., Ke, N., et al.: Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. arXiv preprint arXiv:2410.05229 (2024)
15. Shao, Z., Wang, P., Zhu, Q., Xu, R., et al.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 (2024)
16. Turpin, M., Michael, J., Perez, E., Bowman, S.: Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In: Advances in Neural Information Processing Systems (2023)
17. Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., et al.: Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (2024)
18. Wang, Y., et al.: Beyond reward hacking: Causal rewards for llm alignment. arXiv preprint arXiv:2503.04567 (2025)
19. Wei, J., Wang, X., Schuurmans, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (2022)
20. Weng, Y., Zhu, M., Xia, F., Li, B., et al.: Large language models are better reasoners with self-verification. In: Findings of the Association for Computational Linguistics (2023)
21. Yao, J., Cheng, R., Tan, K.C.: Var-math: Probing true mathematical reasoning in large language models via symbolic multi-instance benchmarks. arXiv preprint arXiv:2507.12885 (2025)
22. Yu, L., Jiang, W., Shi, H., et al.: Metamath: Bootstrap your own mathematical questions for large language models. In: International Conference on Learning Representations (2024)